HW2
(Enigma)

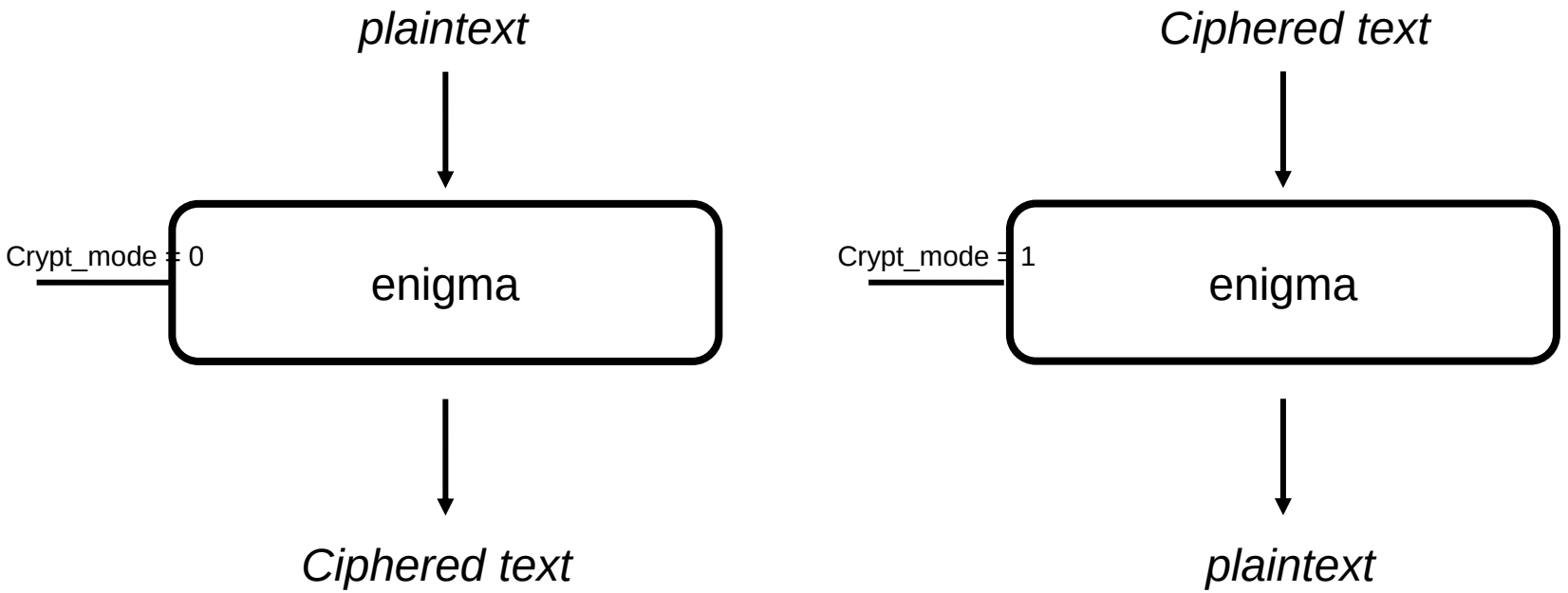
葉紹華

2025.10.02

Enigma

- Enigma
  - Decryption/encryption
  - Characters are mapped to 6bit code

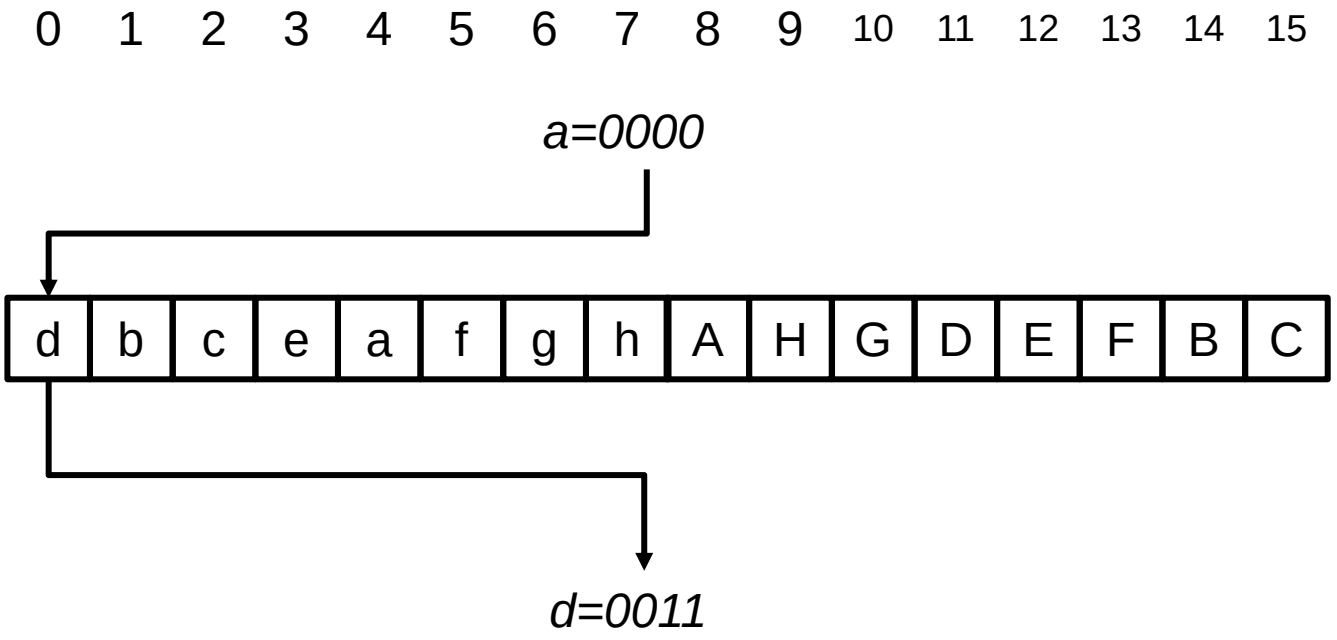
00	01	02	03	04	05	06	07	08	09	0a	0b	0c	0d	0e	0f	10	11	12	13	14	15	16	17	18	19	1a	1b	1c	1d	1e	1f	20	21	22	23	24	25	26	27	28	29	2a	2b	2c	2d	2e	2f	30	31	32	33	34	35	36	37	38	39	3a	3b	3c	3d	3e	3f
a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z		!	,	-	.	\	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	:	#	;	_	+	&



Enigma

a	0	A	8
b	1	B	9
c	2	C	10
d	3	D	11
e	4	E	12
f	5	F	13
g	6	G	14
h	7	H	15

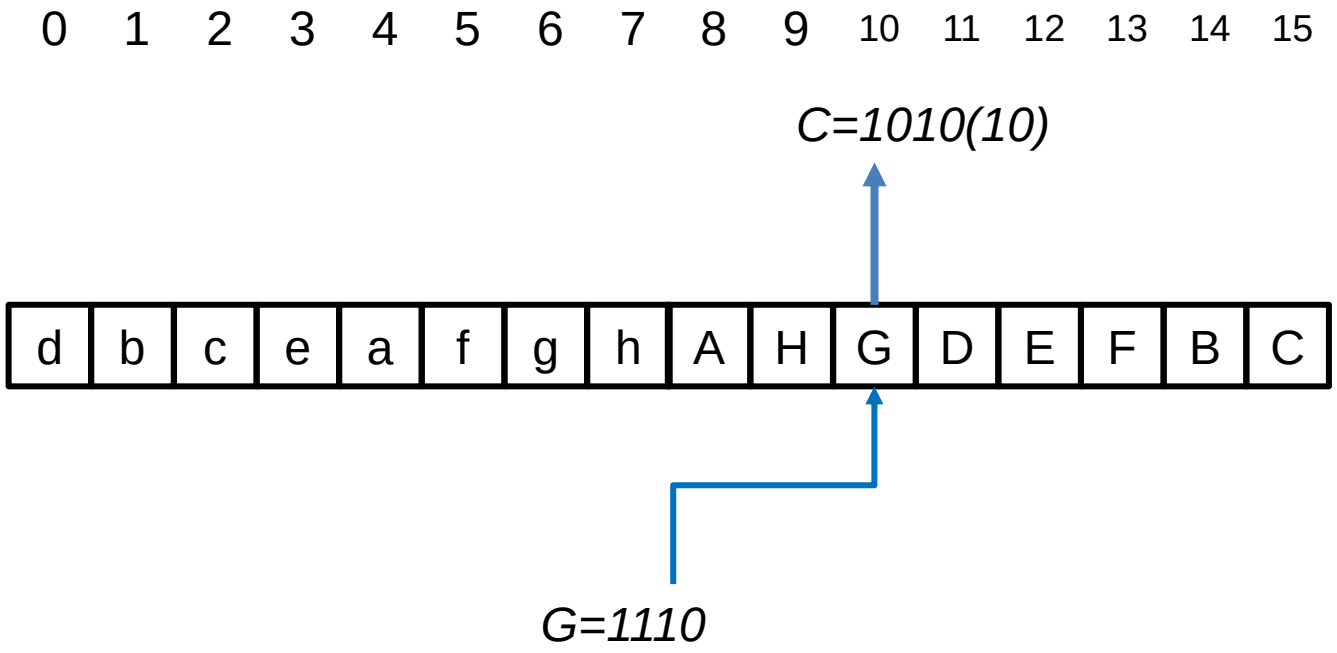
- Rotor
  - Forward path
    - Mapping the input code with the stored code inside rotor



Enigma

a	0	A	8
b	1	B	9
c	2	C	10
d	3	D	11
e	4	E	12
f	5	F	13
g	6	G	14
h	7	H	15

- Rotor
  - inverse path
    - Find the location of the given code inside rotor



Enigma

- Bit-switching box
  - Switch the bits inside code (forward & inverse pass are identical)
    - Mode is determined by least significant 2-bits of
      - Encryption
        - » Output of the forward path
      - Decryption
        - » Input of the inverse path

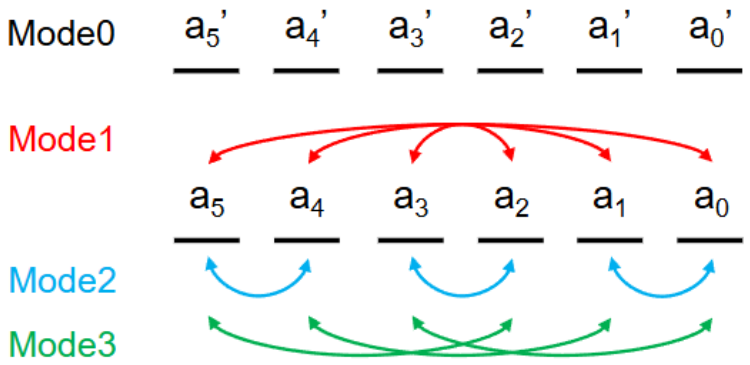


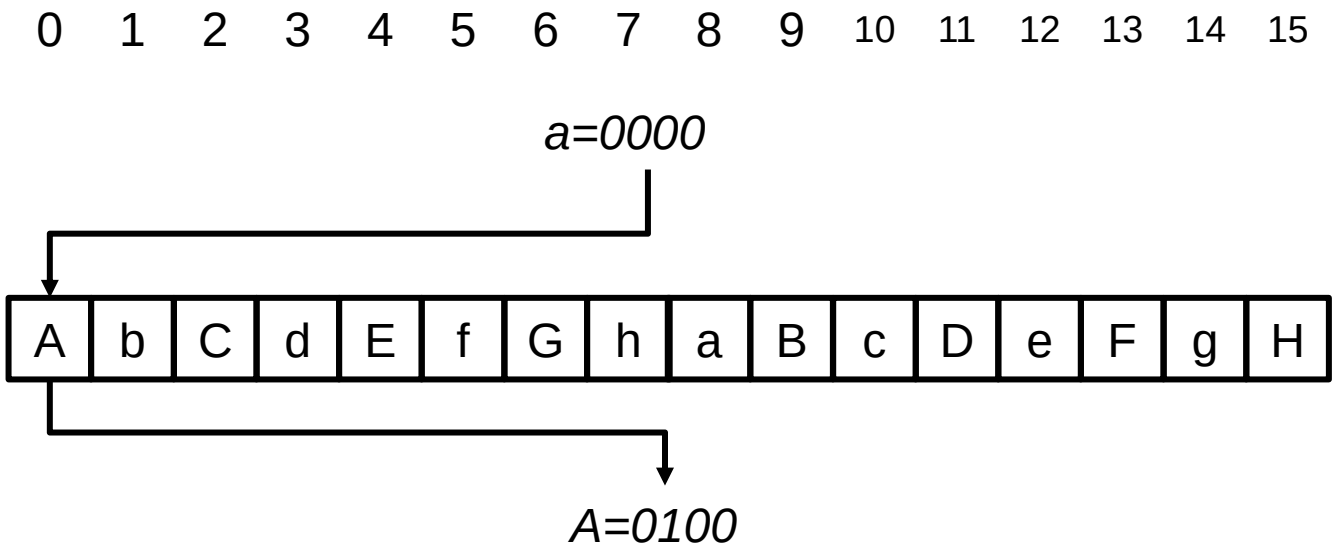
Fig. 6. Bit-switching mode explanation.

$\{a_1, a_0\}$	**Next encryption / decryption**
2'b00	1's complement (Default in the 1st round)
2'b01	Bit-switching by mode1
2'b10	Bit-switching by mode2
2'b11	Bit-switching by mode3

Enigma

a	0	A	8
b	1	B	9
c	2	C	10
d	3	D	11
e	4	E	12
f	5	F	13
g	6	G	14
h	7	H	15

- Plugboard
  - Connect two symbol together
  - For others, output is identical

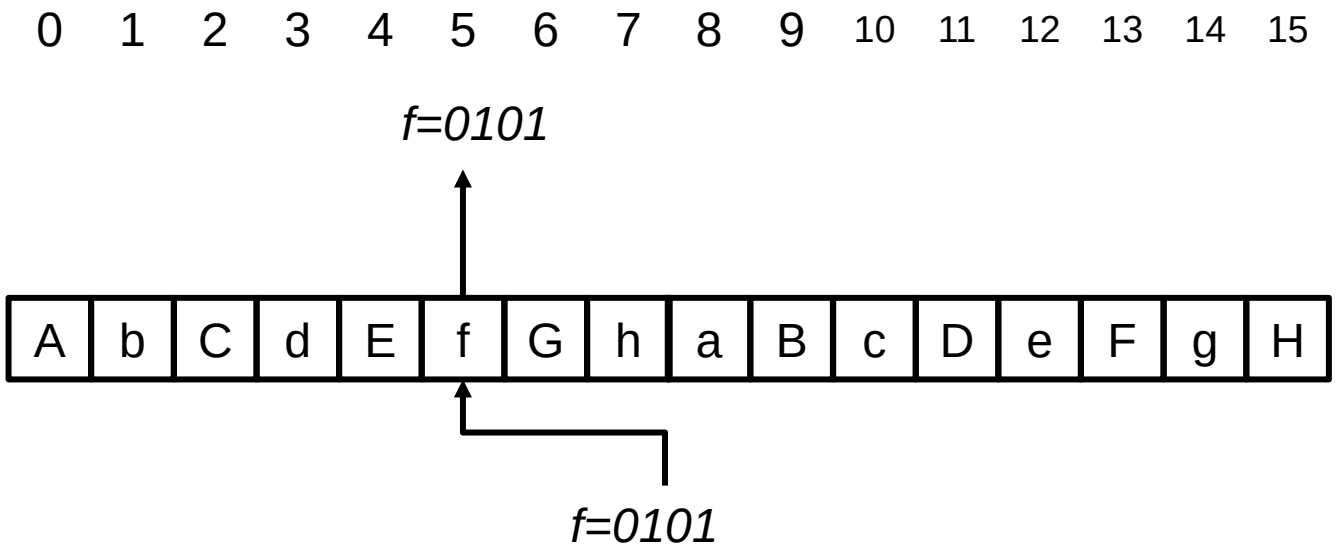


example: four pairs (a,A), (c,C), (e,E), (g,G)

Enigma

a	0	A	8
b	1	B	9
c	2	C	10
d	3	D	11
e	4	E	12
f	5	F	13
g	6	G	14
h	7	H	15

- Plugboard



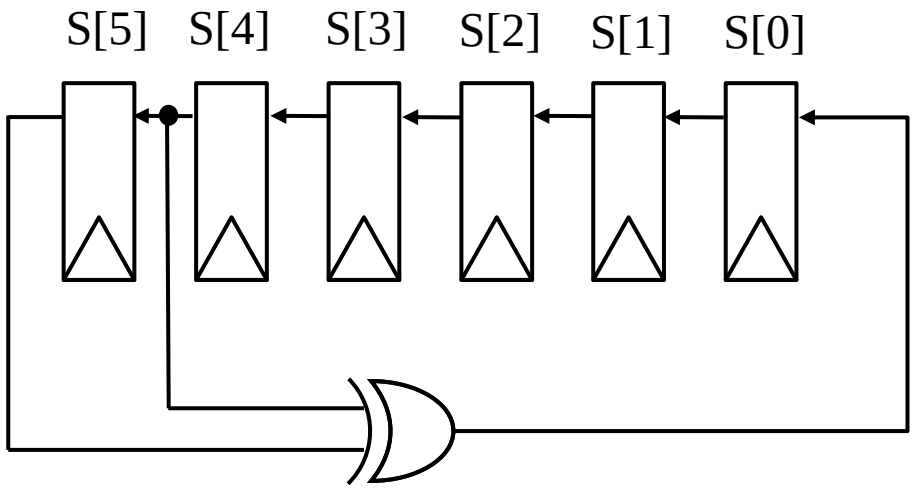
example: four pairs (a,A), (c,C), (e,E), (g,G)

Enigma

cycle	state
1	000001
2	000010
3	000100
4	001000
5	010000
6	100001
7	000011
8	000110
9	001100
10	011000

- XOR whitening
  - Make the input code XOR with current state from LFSR
  - LFSR is given as $x^6 + x^5 + 1$
  - Seed from 000001
  - LFSR shift to the left every time text is XORed with current state

$text[5:0]$ ⊕ $s[5:0]$ ⟵

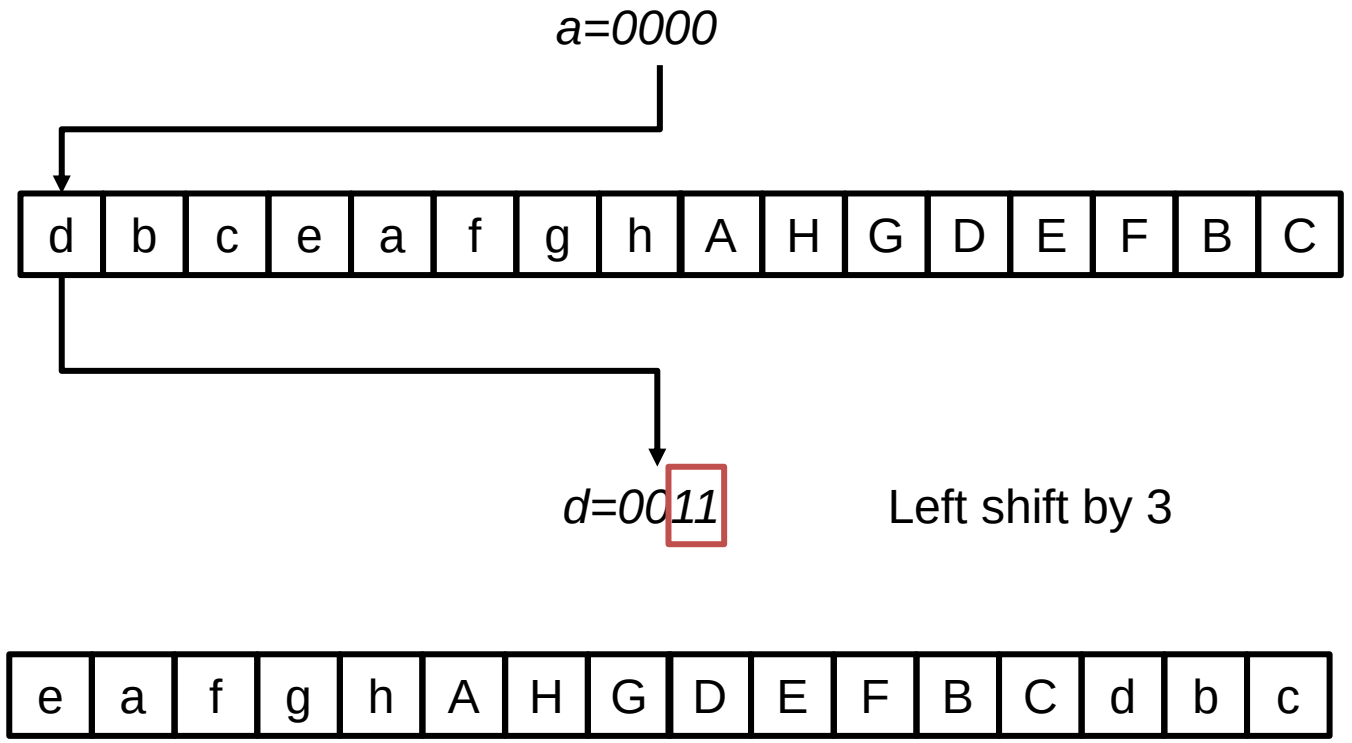


Enigma

- RotorA
  - Will rotate every time a symbol is encrypted/decrypted
  - For rotorA, rotor shift to the left with given shift amount
  - Shift amount is determined by least significant 2bits of
    - Encryption
      - Output of the forward pass
    - Decryption
      - Input of the inverse pass

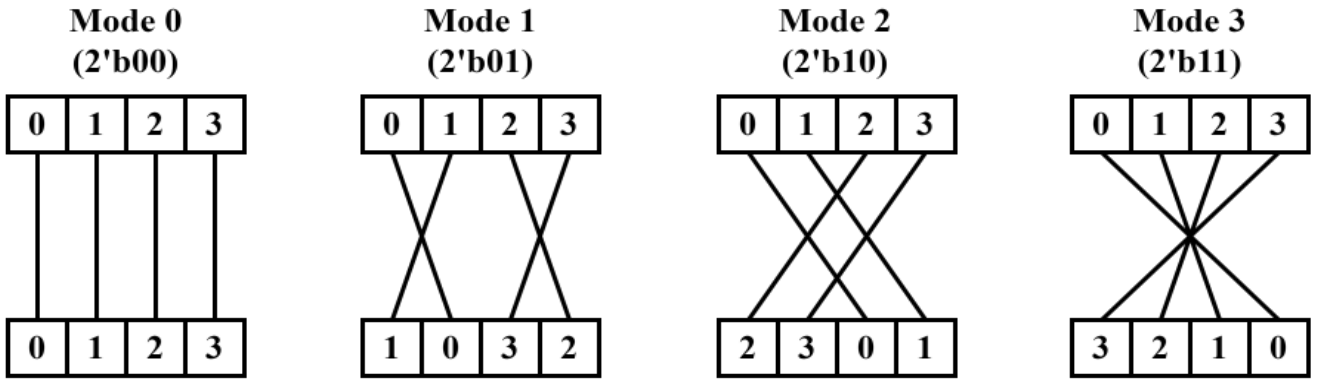
Enigma

- RotorA



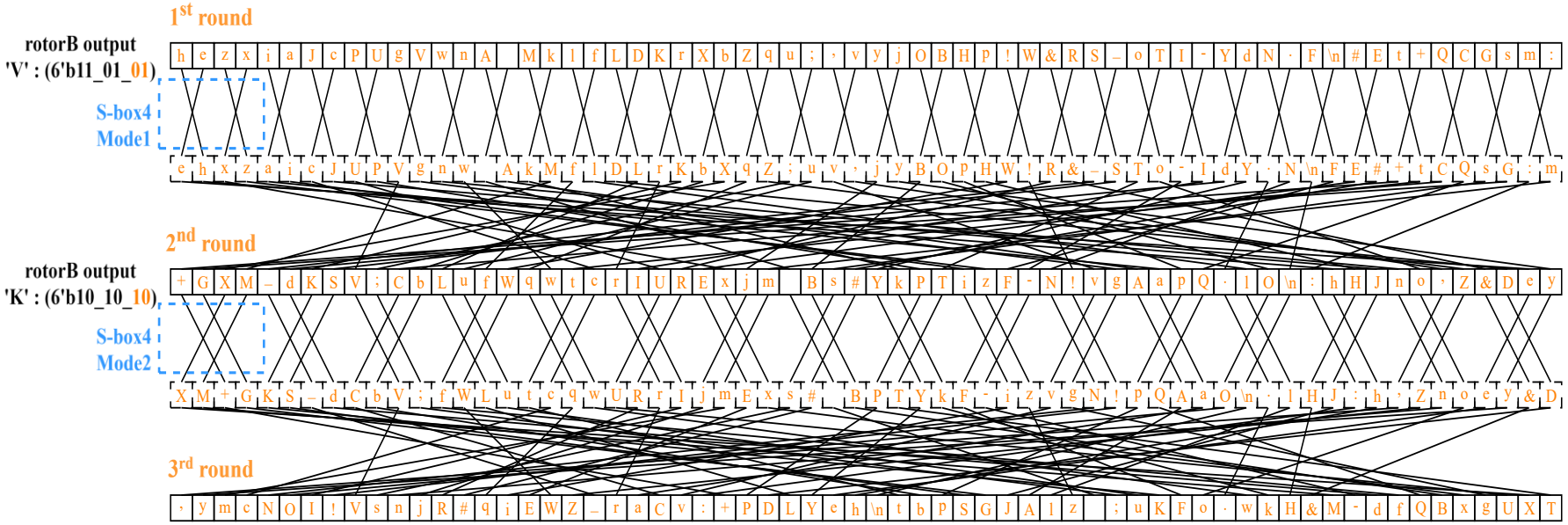
Enigma

- RotorB
  - For rotorB, it goes through two stages
  - Stage1: SBOX, with 4 different modes and is determined by the least significant 2bits of
    - Encryption
      - Output of the forward pass
    - Decryption
      - Input of the inverse pass

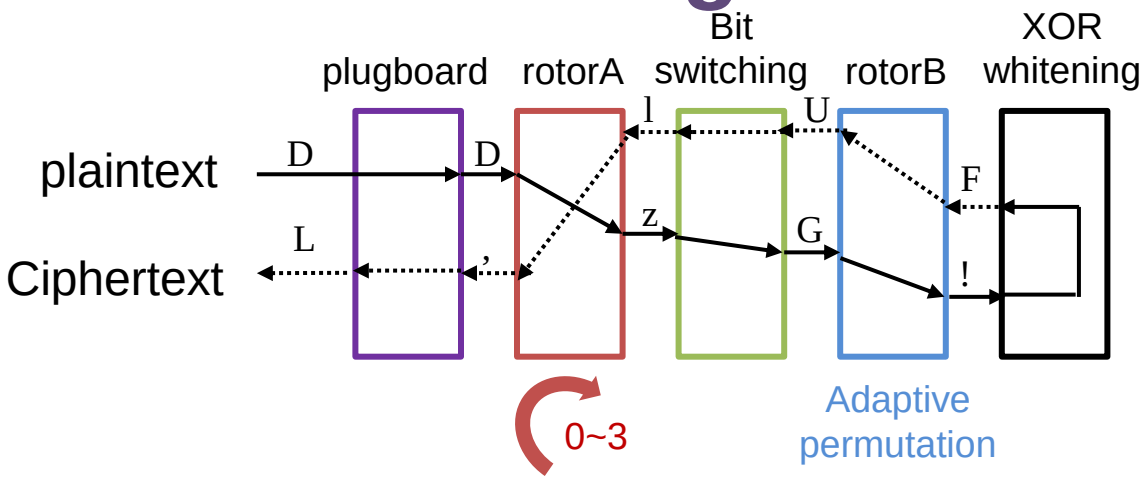


Enigma

- RotorB
  - Stage2: fixed permutation



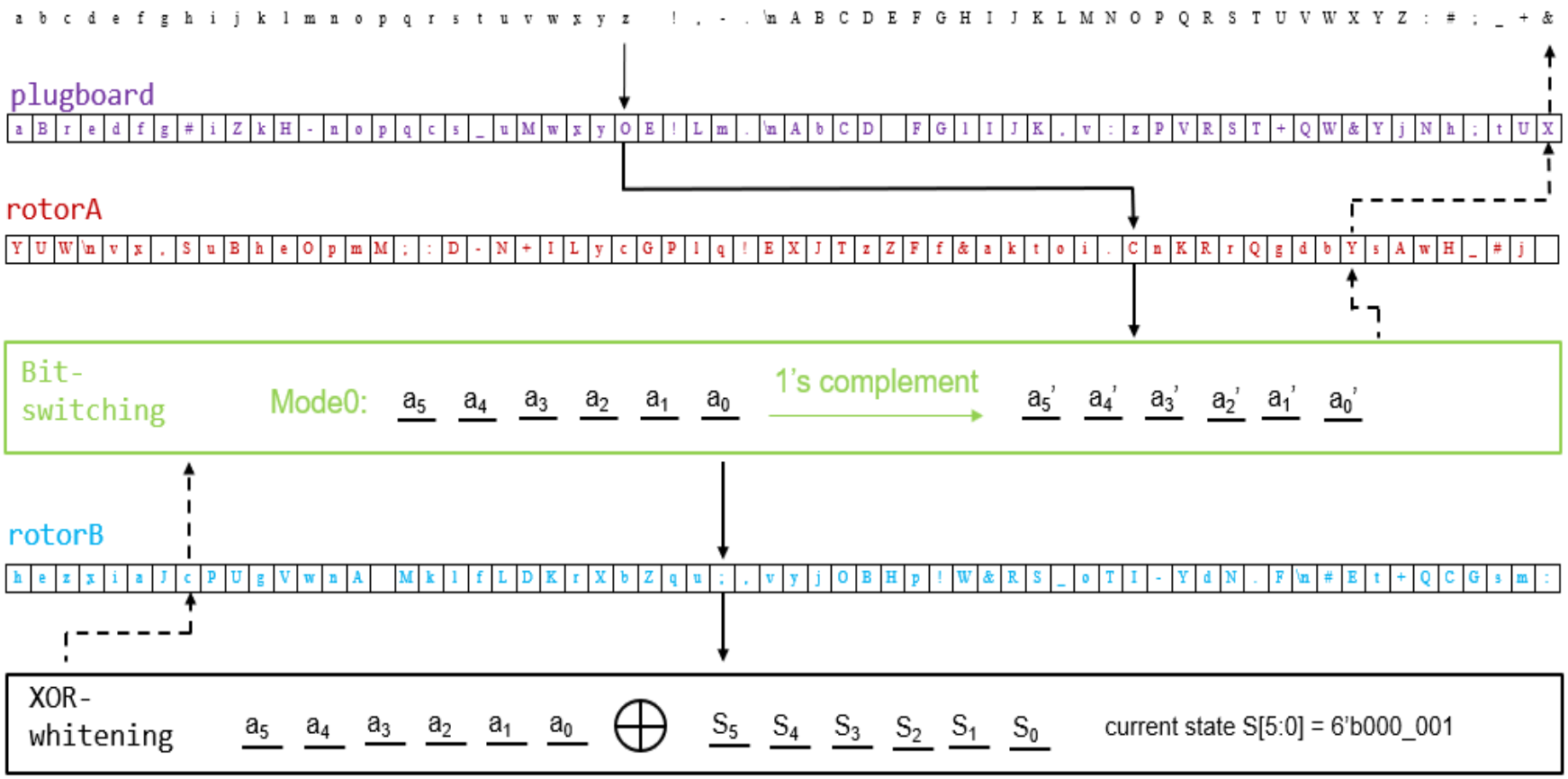
Enigma



Signal	Description
clk	Clock signal
srst_n	Synchronous reset, active low
load	If load is 1'b1, load **code_in** to rotor or plugboard
encrypt	If encrypt is 1'b1, Enigma encrypt **code_in** to **code_out**
crypt_mode	If crypt_mode is 1'b0, **code_in** is plaintext and **code_out** is ciphertext If crypt_mode is 1'b1, **code_in** is ciphertext and **code_out** is plaintext
table_idx[1:0]	If **table_idx** is 2'b00, load **code_in** to rotorA If **table_idx** is 2'b01, load **code_in** to plugboard If **table_idx** is 2'b10, load **code_in** to rotorB
code_in[5:0]	Input code Plaintext if **crypt_mode** is 1'b0 Ciphertext if **crypt_mode** is 1'b1
valid	**valid** is 1'b1 if **code_out** is valid Otherwise, valid is 1'b0
code_out[5:0]	Output code Ciphertext if **crypt_mode** is 1'b0 Plaintext if **crypt_mode** is 1'b1

Enigma

- Example of mapping ‘z’ to ‘&’

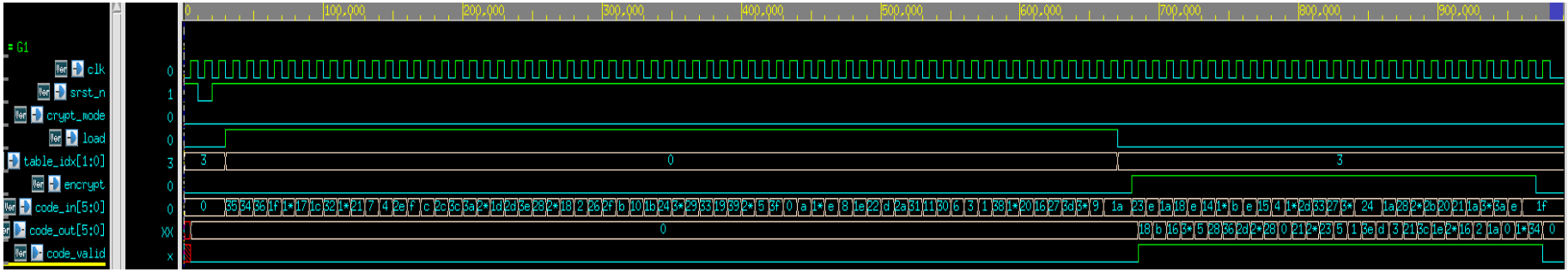


Enigma

- Part1 (20%)
  - RotorA + XOR-whitening
- Part2 (20%)
  - RotorA + bit-switching + XOR-whitening
- Part3 (30%)
  - Plugboard + RotorA+ bit-switching + XOR-whitening
- Part4 (30%)
  - Plugboard + RotorA+ bit-switching + RotorB+ XOR-whitening

Enigma

- Shell file is given in each part
  - Pass all simulation to get full score
  - Waveform is for reference
    - When output valid is high, one output code per cycle
    - Sequence of table_idx might change
    - Don’t hard-code the waveform behavior into circuit
    - Make good use of control signals (load, encrypt, …)



Reference waveform of encrypting pattern 1. The crypt_mode stays at 1’b0. It takes 64 cycles to load rotorA before encrypting.

Enigma

- For part4, an additional macro ASCII is given
  - In decrypted mode, you need to write out the result in ascii format
  - Try to use fopen() fclose() and refer to ASCII table
  - Write the result into sim/result/plaintexts{1,2,3}_ascii.dat

```
# Mode: encrypt; Pattern: PAT1
vcs -R +v2k -full64 -f rtl_sim.f +define+ENCRYPT +define+PAT1 +define+PAT_LEN=29 -debug_access+all -l vcs_encrypt_pat1.log
# Mode: decrypt; Pattern: PAT1
vcs -R +v2k -full64 -f rtl_sim.f +define+DECRYPT +define+PAT1 +define+PAT_LEN=29 +define+ASCII -debug_access+all -l vcs_decrypt_pat1.log

# Mode: encrypt; Pattern: PAT2
vcs -R +v2k -full64 -f rtl_sim.f +define+ENCRYPT +define+PAT2 +define+PAT_LEN=111 -debug_access+all -l vcs_encrypt_pat2.log
# Mode: decrypt; Pattern: PAT2
vcs -R +v2k -full64 -f rtl_sim.f +define+DECRYPT +define+PAT2 +define+PAT_LEN=111 +define+ASCII -debug_access+all -l vcs_decrypt_pat2.log

# # Mode: decrypt; Pattern: PAT3
vcs -R +v2k -full64 -f rtl_sim.f +define+DECRYPT +define+PAT3 +define+PAT_LEN=26465 +define+ASCII -debug_access+all -l vcs_decrypt_pat3.log
```

Enigma

- deliverable

```
HW2_{student_id}\
+---part1
|   +---hdl
|   |       behavior_model.v
|   |       enigma_part1.v
|   |       {xxxx}.v
|   \---sim
|       |   rtl_sim.f
|       |   run_rtl_sim.sh
|       |   test_enigma_part1.v
|       |
|       +---pat
|       |       ciphertexts1_part1.dat
|       |       ciphertexts2_part1.dat
|       |       plaintexts1_part1.dat
|       |       plaintexts2_part1.dat
|       |
|       \---rotor
|               rotorA.dat
+---part2
|   +---hdl
|   |       enigma_part2.v
|   |       {xxxx}.v
|   \---sim
|       |   rtl_sim.f
|       |   run_rtl_sim.sh
|       |   test_enigma_part2.v
|       |
|       +---pat
|       |       ciphertexts1_part2.dat
|       |       ciphertexts2_part2.dat
|       |       plaintexts1_part2.dat
|       |       plaintexts2_part2.dat
|       |
|       \---rotor
|               rotorA.dat
+---part3
|   +---hdl
|   |       enigma_part3.v
|   |       {xxxx}.v
|   \---sim
|       |   rtl_sim.f
|       |   run_rtl_sim.sh
|       |   test_enigma_part3.v
|       |
|       +---pat
|       |       ciphertexts1_part3.dat
|       |       ciphertexts2_part3.dat
|       |       plaintexts1_part3.dat
|       |       plaintexts2_part3.dat
|       |
|       \---rotor
|               plugboard.dat
|               rotorA.dat
\---part4
    +---hdl
    |       enigma_part4.v
    |       {xxxx}.v
    \---sim
        |   rtl_sim.f
        |   run_rtl_sim.sh
        |   test_enigma_part4.v
        |
        +---pat
        |       ciphertexts1_part4.dat
        |       ciphertexts2_part4.dat
        |       ciphertexts3_part4.dat
        |       plaintexts1_part4.dat
        |       plaintexts2_part4.dat
        |       plaintexts3_part4.dat
        |
        +---result
        |       plaintexts1_ascii.dat
        |       plaintexts2_ascii.dat
        |       plaintexts3_ascii.dat
        |
        \---rotor
                plugboard.dat
                rotorA.dat
                rotorB.dat
```

Enigma

- Deadline: 10/16 11:59p.m.
- Submit to eeclass
  - Make sure the file delivery and organization meet the requirement
  - 10 points deducted from this homework for wrong file delivery, wrong file organization, or wrong file naming
  - Delayed submission: $grade_{delayed} = grade \times 0.9^{delayed_{days}}$
- Q&A
  - Feel free to send e-mail to TA (紹華) or post your question on eeclass
  - Do not show your code or email your code to TA. Coding by yourself !
- Using generative AI is not recommended.
  - If used, please explain in the report how it was applied